# `secure_clear`

C Document number: N2682
C++ Document number: P1315R7 [latest]
Date: 2021-03-10
Author: Miguel Ojeda <ojeda@ojeda.dev>
Project: ISO JTC1/SC22/WG14: Programming Language C
Project: ISO JTC1/SC22/WG21: Programming Language C++

# Abstract

Sensitive data, like passwords or keying data, should be cleared from memory as soon as they are not needed. This requires ensuring the compiler will not optimize the memory overwrite away. This proposal adds a `memset_explicit` function (C) and a `memset_explicit` function template (C++) that guarantee users that a memory area is cleared.

# Changelog

## N2682 & P1315R7

- Removed the two wording alternatives not chosen by the committee.
- Removed the Recommended Practice section from Alternative 1 (performance) as polled.
- Added implementation-defined semantics alternative as agreed (Alternative 2).
- Added a variation of both main alternatives (1b and 2b) with details on copies.
- Added `0` as second argument to the C++ function template to match the current C interface.
- Added section on all past polls (both C and C++).
- Added past changelogs.

## N2631 & P1315R6

- Removed the two design alternatives not chosen by the committee.
- Added three wording alternatives.
- Changed “may” to “might” in order to follow the new ISO guidelines.
- Simplified introduction, modified references and other minor changes.

## N2505 & P1315R5

- Merged WG14 C and WG21 C++ proposals.
- The function `secure_clear` is now imported from C; proposal changed accordingly (e.g. removed `noexcept`).

- Updated template declaration to require a constraint on `is_trivially_copyable_v` instead of using the `trivially_copyable` pseudo-concept (since it does not exist in C++20 on its own); and changed `pointer` to `is_pointer_v`.
- Moved the `memset_s` wording suggestion into the C side.
- Removed new header suggestion until discussed with WG14.

### P1315R4

- Constrained the template function to non-pointers, with an added explanation and example.
- Changed spelling of the `TriviallyCopyable` pseudo-concept to `trivially_copyable` following the change in the rest of the standard for all other concepts.

### P1315R3

- Changed to indeterminate values instead of unspecified ones.

### P1315R2

- Removed `secure_val` class template as polled and changed paper title to `secure_clear` to reflect that.
- Added suggestion on wording: lift the “as-if” rule for calls to `secure_clear`, similar to C11’s `memset_s` wording.
- Added intention of working with WG14.
- Split “The problem” section into several.

### P1315R1

- Split `access` member function into `read_access`, `write_access` and `modify_access`.
- Added mention to the SECURE project.

## Past polls

### WG14 March 2021

Does the committee wish to replace the `c` parameter with a specific value from `memset_explicit` in N2631?

```
Y  N  A
4  8 11 => Does not pass.
```

Does the committee wish to replace the `c` parameter with “implementation-defined values” from `memset_explicit` in N2631?

```
 Y  N  A
 4  9  8 => Does not pass.
```

Does the committee prefer Alternatives 1, 2, or 3 as is in N2631?

```
A1 A2 A3
13  4  0 => Alternative 1 wins.
```

Does the committee prefer removing the Recommended Practice section from Alternative 1 in N2631?

```
 Y  N  A
15  0  7 => Passes.
```

Does the committee prefer adding `memset_explicit` to the exception list in `<string.h>` freestanding implementations?

```
Y  N  A
3  8 10 => Does not pass.
```

## WG14/WG21 Liaison March 2021

Should P1315 `memset_explicit` use Alternative 1 plus a statement that the semantics be implementation-defined as a statement of intent rather than an effect on the abstract machine?

```
     SF  F  N  A SA
WG21  5  5  0  0  1 => Consensus.
WG14  2  2  0  0  0 => Consensus.
```

## WG14 November 2020

Does the committee wish to adopt something along the lines of alternative 3 of N2599 into C23?

```
 Y  N  A
16  1  6 => Clear direction.
```

## WG14 August 2020

Would the Committee like to see a non-elidable, non-optional memory-erasing function added to C2x?

```
 Y  N  A
14  0  2 => Clear direction.
```

Would the Committee like the non-elidable, non-optional memory-erasing function not to specify a value in its interface?

```
 Y  N  A
 6  5  6 => Unclear direction.
```

Would the Committee like to be able to specify a value in the interface to the non-elidable, non-optional memory-erasing function?

```
 Y  N  A
 7  4  6 => Clearer direction.
```

Would the Committee like to have both no-value and value-specifying interfaces to the non-elidable, non-optional function available?

```
 Y  N  A
 5  6  7 => Unclear direction.
```

## WG21 SG1 November 2019

We don't see a specific SG1 concern for what we presume is a single-threaded API. If this paper intends to make `secure_clear` resilient to UB in C++ then data-race UB is but one of the kinds.

```
No objection to unanimous consent.
```

## WG21 EWGI July 2019

Spend committee time on this versus other proposals, given that time is limited?

```
SF  F  N  A SA
 2  9  2  1  0 => Consensus.
```

Send the paper to SG1 for input on abstract machine integration and wording (similar to `volatile_load`/`volatile_store`). Send it back to us after.

```
SF  F  N  A SA
 4  5  4  0  0 => Consensus.
```

## WG21 EWGI February 2019

Remove all cache related things from the proposal.

```
SF  F  N  A SA
 3  1  3  0  0 => Consensus.
```

Remove encrypting at rest from the proposal.

```
SF  F  N  A SA
 4  1  1  1  0 => Consensus.
```

Want `secure_clear` to write indeterminate values (as opposed to `memset_s`).

```
SF  F  N  A SA
 4  1  2  0  0 => Consensus.
```

Want to work with WG14 on `secure_clear` (e.g. salvage `memset_s` from Annex K).

```
SF  F  N  A SA
 2  3  2  0  0 => Consensus.
```

We want something along the lines of `secure_val` (with compiler support).

```
SF  F  N  A SA
 0  0  2  2  3 => Consensus to reject.
```

# The problem

When manipulating sensitive data, like passwords in memory or keying data, there is a need for library and application developers to clear the memory after the data is not needed anymore [1][2][3][4], in order to minimize the time window where it is possible to capture it (e.g., ending in a core dump or probed by a malicious actor). Recent vulnerabilities (e.g., Meltdown, Spectre-class, Rowhammer-class…) have made this requirement even more prominent.

To ensure that the memory is cleared, the developer needs to inform the compiler that it must not optimize away the memory write, even if it can prove it has no observable behavior. For C++, extra care is needed to consider all exceptional return paths.

For instance, the following C function may be vulnerable, since the compiler may optimize the `memset` call away because the `password` buffer is not read from before it goes out of scope:

```
void f(void)
{
    char password[SIZE];

    // Acquire some sensitive data
    get_password_from_user(password, SIZE);

    // Use it for some operations
    use_password(password, SIZE);

    // Attempt to clear the sensitive data
    memset(password, 0, SIZE);
}
```

On top of that, in C++, `use_password` could throw an exception so `memset` is never called (i.e., assume the stack is not overwritten and/or that the memory is held in the free store).

There are many ways that developers may use to try to ensure the memory is cleared as expected (i.e., avoiding the optimizer):

- Calling `memset_s` from Annex K, if available.
- Calling a function defined in another translation unit, assuming LTO/WPO is not enabled.
- Writing memory through a `volatile` pointer (e.g., `decaf_bzero` [5] from OpenSSL).
- Calling `memset` through a `volatile` function pointer (e.g., `OPENSSL_cleanse` C implementation [6]).
- Creating a dependency by writing an `extern` variable into it (e.g., `CRYPTO_malloc` implementation [7] from OpenSSL).
- Coding the memory write in assembly (e.g., `OPENSSL_cleanse` SPARC implementation [8]).
- Introducing a memory barrier (e.g., `memzero_explicit` implementation [9] from the Linux Kernel).
- Disabling specific compiler options/optimizations (e.g., `-fno-builtin-memset` [10] in GCC).

Or they may use a pre-existing solution whenever available:

- Using an operating system-provided API (e.g., `explicit_bzero` [11] from OpenBSD & FreeBSD, `SecureZeroMemory` [12] from Windows).
- Using a library function (e.g., `memzero_explicit` [13][9] from the Linux Kernel, `OPENSSL_cleanse` [14][6]).

Regardless of how it is done, none of these ways is — at the same time — portable, easy to recognize the intent (and/or `grep` for it), readily available and avoiding compiler

implementation details. The topic may generate discussions in major projects on which is the best way to proceed and whether an specific approach ensures that the memory is actually cleansed (e.g., [15][16][17][18][19]). Sometimes, such a way is not effective for a particular compiler (e.g., [20]). In the worst case, bugs happen (e.g., [21][22]).

C11 (and C++17 as it was based on it) added `memset_s` (K.3.7.4.1) to give a standard solution to this problem [4][23]. However, it is an optional extension (Annex K) and, at the time of writing, several major compiler vendors do not define `__STDC_LIB_EXT1__` (GCC [24], Clang [25], MSVC [26], icc [27]). Therefore, in practical terms, there is no widely-available, portable solution yet for C nor C++. A 2015 paper on this topic, WG14's N1967 "Field Experience With Annex K — Bounds Checking Interfaces" [28], concludes that "Annex K should be removed from the C standard". On the other hand, a 2019 paper, WG14's N2336 "Bounds-checking Interfaces: Field Experience and Future Directions" [29], examines the arguments both for and against the bounds-checked interfaces, evaluates possible solutions for actual problems with Annex K, and propounds the repair and retention of Annex K for C2x.

Other languages offer similar facilities in their standard libraries or as external projects:

- Limited private types in Ada and SPARK [30][31].
- The `secrecy`, `secstr` and `zeroize` libraries in Rust [32][33][34].
- The `SecureString` class in .NET Core, .NET Framework and Xamarin [35].
- The `securemem` package in Haskell [36].

# The basic solution

We can standardize current practise by providing a `memset_explicit` function that takes a memory range (a pointer and a size) to be erased, plus a value, guaranteeing that it won't be optimized away:

```
memset_explicit(password, 0, size);
```

As well as a `memset_explicit` function template for C++ that takes a reference to a non-pointer `trivially_copyable` object to scrub it entirely:

```
std::memset_explicit(password);
```

There are several reasonable design alternatives for such a function with respect to its interface, semantics, wording, naming, etc. Both WG14 and WG21 discussed them in several instances on previous revisions of this proposal. In the end, the C committee decided to go for a function with the same interface as `memset` for simplicity and familiarity.

# Further issues and improvements

While it is a good practise to ensure that the memory is cleared as soon as possible, there are other potential improvements when handling sensitive data in memory:

- Reducing the number of copies to a minimum.
- Clearing registers, caches, the entire stack frame, etc.
- Locking/pinning memory to avoid the data being swapped out to external storage (e.g., disk).
- Encrypting the memory while it is not being accessed to avoid plain-text leaks (e.g., to a log or in a core dump).
- Turning off speculative execution (or mitigating it). At the time of writing, Spectre-class vulnerabilities are still being fixed (either in software or in hardware), and new ones are coming up [37].
- Techniques related to control flow, e.g., control flow balancing (making all branches spend the same amount of time).

Most of these extra improvements require either non-portable code, operating-system-specific calls, or compiler support as well, which also makes them a good target for standardization. A subset (e.g., encryption-at-rest and locking/pinning) may be relatively easy to tackle in the near future since libraries/projects and other languages handle them already. Other improvements, however, are well in the research stage on compilers, e.g.:

- The SECURE project and GCC (stack erasure implemented as a function attribute) [38] [39][40].
- Research on controlling side effects in mainstream C compilers [41].

Furthermore, discussing the addition of any security-related features to the C and C++ languages is a complex task. Therefore, this paper attempts to only spearhead the work on this area, providing access to users to the well-known “guaranteed memory clearing” function already used in the industry as explained in the previous sections. For context, it is important to mention that, recently, there seems to be a resurgence of interest in these topics: for C, N2485 proposed to add similar functionality [42]; while within C++ there are ongoing discussions about creating a safety and security SG or SIG.

Some other related improvements based on the basic building blocks can be also thought of, too:

- For instance, earlier revisions of the C++ paper [43] proposed, in addition, an uncopyable class template meant to wrap a memory area on which `memset_explicit` is called on destruction/move, plus some other features (explicit read/modify/write access, locking/pinning, encryption-at-rest, etc.). This wrapper is a good approach to ensure

memory is cleared even when dealing with exceptions. During the WG21 LEWGI review, it was acknowledged that similar types are in use by third-parties. Some libraries and languages feature similar types, as mentioned in the previous section.
- A type-modifier with similar semantics as the wrapper class template above was suggested during the WG21 LEWGI review; as well as other approaches like new keywords to mark variables to be cleared at the end of scope.
- Dynamically-sized string-like types (e.g., `secure_string`) may be considered useful (e.g., for passwords).

There are, as well, other related library features that some languages provide for handling sensitive data:

- A way to read non-echoed standard input (e.g., see the `getpass` module in Python [44]).

# Proposal (C)

This proposal adds the `memset_explicit` function which follows the `memset` interface, semantics and naming.

Two main wording alternatives are proposed (1 and 2). Alternative 1 is the same as in N2631. Alternative 2 converts the semantics into implementation-defined.

Each main alternative has a variation (1b and 2b) that adds extra details on the potential clearing and avoidance of copies by implementations.

The proposed wordings are with respect to N2596 C2x Working Draft.

## Alternative 1: Intent semantics

After “7.24.6.1 The `memset` function”, add:

> **7.24.6.2 The `memset_explicit` function**
>
> **Synopsis**
>
> ```
> #include <string.h>
> void *memset_explicit(void *s, int c, size_t n);
> ```
>
> **Description**
>
> The `memset_explicit` function copies the value of `c` (converted to an `unsigned char`) into each of the first `n` characters of the object pointed to by `s`. The purpose of this function is to make sensitive information stored in the object inaccessible. (Footnote 1)

(Footnote 1) The intention is that the memory store is always performed (i.e., never elided), regardless of optimizations. This is in contrast to calls to the `memset` function (7.24.6.1).

**Returns**

The `memset_explicit` function returns the value of `s`.

In “B.23 String handling `<string.h>`”, after the `memset` line, add:

```
void *memset_explicit(void *s, int c, size_t n);
```

In “J.6.1 Rule based identifiers”, after the `memset` line, add:

```
memset_explicit
```

## Alternative 1b: Intent semantics + details on copies

After “7.24.6.1 The `memset` function”, add:

### 7.24.6.2 The `memset_explicit` function

**Synopsis**

```
#include <string.h>
void *memset_explicit(void *s, int c, size_t n);
```

**Description**

The `memset_explicit` function copies the value of `c` (converted to an `unsigned char`) into each of the first `n` characters of the object pointed to by `s`. The purpose of this function is to make sensitive information stored in the object inaccessible. (Footnote 1)

(Footnote 1) The intention is that the memory store is always performed (i.e., never elided), regardless of optimizations. This is in contrast to calls to the `memset` function (7.24.6.1). The implementation might clear other copies of the data (e.g., intermediate values, stack frame, cache lines, spilled registers, swapped out pages, etc.) or it might avoid their creation (e.g., reducing copies, locking/pinning pages, etc.).

**Returns**

The `memset_explicit` function returns the value of `s`.

In “B.23 String handling `<string.h>`”, after the `memset` line, add:

```
void *memset_explicit(void *s, int c, size_t n);
```

In “J.6.1 Rule based identifiers”, after the `memset` line, add:

```
memset_explicit
```

## Alternative 2: Implementation-defined semantics

After “7.24.6.1 The `memset` function”, add:

### 7.24.6.2 The `memset_explicit` function

#### Synopsis

```
#include <string.h>
void *memset_explicit(void *s, int c, size_t n);
```

#### Description

The semantics of the `memset_explicit` function are implementation-defined.

#### Recommended Practice

Implementations should copy the value of `c` (converted to an `unsigned char`) into each of the first `n` characters of the object pointed to by `s` in order to make sensitive information stored in the object inaccessible. (Footnote 1)

(Footnote 1) The intention is that the memory store is always performed (i.e., never elided), regardless of optimizations. This is in contrast to calls to the `memset` function (7.24.6.1).

#### Returns

The `memset_explicit` function returns the value of `s`.

In “B.23 String handling `<string.h>`”, after the `memset` line, add:

```
void *memset_explicit(void *s, int c, size_t n);
```

In “J.6.1 Rule based identifiers”, after the `memset` line, add:

```
memset_explicit
```

## Alternative 2b: Implementation-defined semantics + details on copies

After “7.24.6.1 The `memset` function”, add:

> **7.24.6.2 The `memset_explicit` function**
>
> **Synopsis**
>
> ```
> #include <string.h>
> void *memset_explicit(void *s, int c, size_t n);
> ```
>
> **Description**
>
> The semantics of the `memset_explicit` function are implementation-defined.
>
> **Recommended Practice**
>
> Implementations should copy the value of `c` (converted to an `unsigned char`) into each of the first `n` characters of the object pointed to by `s` in order to make sensitive information stored in the object inaccessible. (Footnote 1)
>
> (Footnote 1) The intention is that the memory store is always performed (i.e., never elided), regardless of optimizations. This is in contrast to calls to the `memset` function (7.24.6.1). The implementation might clear other copies of the data (e.g., intermediate values, stack frame, cache lines, spilled registers, swapped out pages, etc.) or it might avoid their creation (e.g., reducing copies, locking/pinning pages, etc.).
>
> **Returns**
>
> The `memset_explicit` function returns the value of `s`.

In “B.23 String handling `<string.h>`”, after the `memset` line, add:

```
void *memset_explicit(void *s, int c, size_t n);
```

In “J.6.1 Rule based identifiers”, after the `memset` line, add:

```
memset_explicit
```

# Proposal (C++)

This proposal adds the `memset_explicit` function template which would rely on the C2x function:

```
namespace std {
    template <class T>
        requires is_trivially_copyable_v<T>
            and (not is_pointer_v<T>)
    void memset_explicit(T & object) noexcept;
}
```

The `memset_explicit` function template is equivalent to a call to `memset_explicit(addressof(object), 0, sizeof(object))`.

The `not is_pointer_v<T>` constraint is intended to prevent unintended calls that would have cleared a pointer rather than the object it points to:

```
char buf[size];
char * ptr = buf;

std::memset_explicit(buf);               // OK, clears the array
std::memset_explicit(ptr);               // Error
std::memset_explicit(ptr, sizeof(ptr));  // OK, explicit
```

No wording is provided yet, since so far the C function has not been added to the C2x Working Draft nor the C++2y standard has updated its normative references section to import the C2x standard library.

# Example implementation

A trivial example implementation (i.e., relying on OS-specific functions) can be found at [45].

# Acknowledgements

Thanks to Aaron Ballman, JF Bastien, David Keaton and Billy O'Neal for providing guidance about the WG14 and WG21 standardization processes. Thanks to Aaron Ballman, Peter Sewell, David Svoboda, Hubert Tong, Martin Uecker, Ville Voutilainen and others for their input on wording and the abstract machine. Thanks to Aaron Peter Bachmann for withdrawing his parallel proposal [42]. Thanks to Robert C. Seacord for his review and proofreading of a previous revision. Thanks to Ryan McDougall for presenting an early revision at WG21 Kona 2019. Thanks to Graham Markall for his input regarding the SECURE project and the current state of compiler support for related features. Thanks to Martin Sebor for pointing out the SECURE project. Thanks to BSI for suggesting constraining the template to non-pointers. Thanks to Philipp Klaus Krause for raising the discussion in the OpenBSD list. Thanks to everyone else that joined all the different discussions.

# References


1. SEI CERT C Coding Standard. “MSC06-C. Beware of compiler optimizations” — *https://wiki.sei.cmu.edu/confluence/display/c/MSC06-C.+Beware+of+compiler+optimizations*
2. Common Weakness Enumeration. “CWE-14: Compiler Removal of Code to Clear Buffers” — *https://cwe.mitre.org/data/definitions/14.html*
3. Viva64. “V597. The compiler could delete the `memset` function call (…)” — *https://www.viva64.com/en/w/v597/*
4. David Svoboda. “WG14 N1381 #5 `memset_s()` to clear memory, without fear of removal” — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1381.pdf*
5. OpenSSL. “`openssl/crypto/ec/curve448/utils.c` (old code)” — *https://github.com/openssl/openssl/blob/f8385b0fc0215b378b61891582b0579659d0b9f4/crypto/ec/curve448/utils.c*
6. OpenSSL. “`OPENSSL_cleanse` (implementation)” — *https://github.com/openssl/openssl/blob/master/crypto/mem_clr.c*
7. OpenSSL. “`openssl/crypto/mem.c` (old code)” — *https://github.com/openssl/openssl/blob/104ce8a9f02d250dd43c255eb7b8747e81b29422/crypto/mem.c#L143*
8. OpenSSL. “`openssl/crypto/sparccpuid.S` (example of assembly implementation)” — *https://github.com/openssl/openssl/blob/master/crypto/sparccpuid.S#L363*
9. Linux Kernel. “`memzero_explicit` (implementation)” — *https://elixir.bootlin.com/linux/v4.18.5/source/lib/string.c#L706*
10. GCC. “Options Controlling C Dialect” — *https://gcc.gnu.org/onlinedocs/gcc/C-Dialect-Options.html*
11. Linux man-pages. “`bzero`, `explicit_bzero` - zero a byte string” — *http://man7.org/linux/man-pages/man3/bzero.3.html*
12. Microsoft. “`SecureZeroMemory` function” — *https://msdn.microsoft.com/en-us/library/windows/desktop/aa366877(v=vs.85).aspx*
13. Linux Kernel. “`memzero_explicit`” — *https://www.kernel.org/doc/htmldocs/kernel-api/API-memzero-explicit.html*
14. OpenSSL. “`OPENSSL_cleanse`” — *https://www.openssl.org/docs/man1.1.1/man3/OPENSSL_cleanse.html*
15. OpenSSL. “Reimplement non-asm `OPENSSL_cleanse()` #455” — *https://github.com/openssl/openssl/pull/455*
16. Colin Percival. “How to zero a buffer” — *http://www.daemonology.net/blog/2014-09-04-how-to-zero-a-buffer.html*
17. Colin Percival et. al.. “How to zero a buffer (daemonology.net)” — *https://news.ycombinator.com/item?id=8270136*
18. Daniel Trebbien et. al.. “Mac OS X equivalent of `SecureZeroMemory` / `RtlSecureZeroMemory`?” — *https://stackoverflow.com/questions/13299420/*
19. Sandy Harris. “Optimising away `memset()` calls?” — *https://gcc.gnu.org/ml/gcc-help/2014-10/msg00047.html*
20. Felix von Leitner. “Bug 15495 - dead store pass ignores memory clobbering asm statement” — *https://bugs.llvm.org/show_bug.cgi?id=15495*
21. zatimend. “Bug 82041 - `memset` optimized out in `random.c`” — *https://bugzilla.kernel.org/show_bug.cgi?id=82041*

22. Daniel Borkmann. “[PATCH] random: add and use memzero_explicit() for clearing data” — *https://lore.kernel.org/lkml/1408996899-4892-1-git-send-email-dborkman@redhat.com/*
23. Larry Jones. “WG14 N1548 Working Draft with diff marks against N1256” — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1548.pdf*
24. Miguel Ojeda. “Test for `memset_s` in gcc 8.2 at Godbolt” — *https://godbolt.org/g/M7MyRg*
25. Miguel Ojeda. “Test for `memset_s` in clang 6.0.0 at Godbolt” — *https://godbolt.org/g/ZwbkgY*
26. Miguel Ojeda. “Test for `memset_s` in MSVC 19 2017 at Godbolt” — *https://godbolt.org/g/FtrVJ8*
27. Miguel Ojeda. “Test for `memset_s` in icc 18.0.0 at Godbolt” — *https://godbolt.org/g/vHZNrW*
28. Carlos O’Donell, Martin Sebor. “WG14 N1967 Field Experience With Annex K - Bounds Checking Interfaces” — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1967.htm*
29. Robert C. Seacord. “WG14 N2336 Bounds-checking Interfaces: Field Experience and Future Directions” — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2336.pdf*
30. Roderick Chapman. “Sanitizing Sensitive Data: How to get it Right (or at least Less Wrong…)” — *https://proteancode.com/wp-content/uploads/2017/06/ae2017_final3_for_web.pdf*
31. Roderick Chapman. “Sanitizing Sensitive Data: How to get it Right (or at least Less Wrong…)” — *https://link.springer.com/chapter/10.1007%2F978-3-319-60588-3_3*
32. Tony Arcieri. “`secrecy` Rust library” — *https://crates.io/crates/secrecy*
33. Greg V. “`secstr` Rust library” — *https://crates.io/crates/secstr*
34. Tony Arcieri. “`zeroize` Rust library” — *https://crates.io/crates/zeroize*
35. Microsoft. “`SecureString` Class” — *https://docs.microsoft.com/en-us/dotnet/api/system.security.securestring*
36. Vincent Hanquez. “`securemem`: abstraction to an auto scrubbing and const time eq, memory chunk.” — *https://hackage.haskell.org/package/securemem*
37. Chandler Carruth. “CppCon 2018 “Spectre: Secrets, Side-Channels, Sandboxes, and Security” — *https://www.youtube.com/watch?v=_f7O3IfIR2k*
38. Graham Markall. “The SECURE Project and GCC” — *https://www.youtube.com/watch?v=Lg_jJLtH7R0*
39. Graham Markall. “The SECURE project and GCC” — *https://gcc.gnu.org/wiki/cauldron2018#secure*
40. Embecosm. “The Security Enhancing Compilation for Use in Real Environments (SECURE) project” — *https://www.embecosm.com/research/secure/*
41. Laurent Simon, David Chisnall, Ross Anderson. “What you get is what you C: Controlling side effects in mainstream C compilers” — *https://www.cl.cam.ac.uk/~rja14/Papers/whatyouc.pdf*
42. Aaron Peter Bachmann. “WG14 N2485 Add explicit_memset() as non-optional part of to C2X” — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n2485.htm*
43. Miguel Ojeda. “WG21 P1315R1 `secure_val`: a secure-clear-on-move type” — *http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1315r1.html*
44. Python. “Portable password input” — *https://docs.python.org/3.7/library/getpass.html*
45. Miguel Ojeda. “`secure_clear` Example implementation” — *https://github.com/ojeda/secure_clear/tree/master/example-implementation*